Phase 3 Report: End-to-End Machine Learning Pipeline

FDA FAERS Adverse Event Seriousness Prediction

Student: Hasti

Data Science Course Project

Deadline: 28th Tir

Instructors: Dr. Razavi, Dr. Yaghoobzadeh

Teaching Assistants:
Mina Shirazi, Mostafa Kermani Nia, Mehrad Livian, Mohammad Amanlou, Mohammad Reza Mohammad Hashemi, Yasaman Amou Jafary

A leakage-free binary classification system with graph-based drug features, fold-safe model selection, hyperparameter tuning, and Prefect automation.

Contents

1 Introduction 4

2 Project Objectives 4

3 Dataset and Relational Database 5

4 Task Definition and Input Features 5
- 4.1 Target Variable . . . 5
- 4.2 Base Features . . . 5
- 4.3 Graph-Based Features . . . 6

5 Graph-Based Drug Risk Feature Engineering 6
- 5.1 Drug Co-occurrence Graph . . . 6
- 5.2 PageRank Risk Score . . . 6
- 5.3 Leakage Prevention . . . 7

6 Training, Validation, and Test Strategy 7

7 Candidate Models 8

8 Cross-Validation Model Comparison 8

9 Hyperparameter Tuning 9

10 Final Model Evaluation 10
- 10.1 Evaluation Metrics . . . 10
- 10.2 Held-Out Test Results . . . 11
- 10.3 Confusion Matrix . . . 11
- 10.4 ROC Curve . . . 12

11 Feature Importance 12

12 Training Pipeline Automation with Prefect 14
- 12.1 Reason for Selecting Prefect . . . 14
- 12.2 Training Flow Tasks . . . 14
- 12.3 Training Flow Dependencies . . . 15

13 Final Model Artifact and Saved Outputs 15

14 Prediction Pipeline 16
- 14.1 Prediction Tasks . . . 16
- 14.2 Prediction Input Preparation . . . 16
- 14.3 Predicted Values . . . 17

15 Saving Predictions Back to SQLite 17

16 End-to-End Execution 17

17 Reproducibility 18

18 Discussion **19**

19 Limitations **19**

20 Future Work **20**

21 Conclusion **20**

1 Introduction

The purpose of Phase 3 was to complete the data science workflow by developing, evaluating, and integrating a machine learning model into an automated end-to-end pipeline. The project uses data derived from the FDA Adverse Event Reporting System (FAERS), which contains reports of adverse events associated with drugs and medical products.

The main prediction task is to determine whether an adverse event report is serious or non-serious. Therefore, the problem is formulated as a binary classification task, where the target variable is defined as:

- **serious = 1**: the adverse event report is serious;
- **serious = 0**: the adverse event report is non-serious.

The developed system combines conventional patient-level and report-level variables with graph-based features extracted from a drug co-occurrence network. Multiple classification algorithms were compared through fold-safe stratified cross-validation. The best-performing algorithm was selected automatically, optimized using randomized hyperparameter search, and evaluated on a completely held-out test set.

Two separate Prefect workflows were developed:

1. a **training pipeline** for database construction, preprocessing, feature engineering, model comparison, and final model training;
2. a **prediction pipeline** for loading database records, applying the saved model, and storing predictions back in SQLite.

2 Project Objectives

The main objective was to create a reproducible and automated prediction system for adverse event seriousness. The specific objectives were:

1. Load and organize FAERS data in a relational SQLite database.
2. Preprocess patient, report, drug, and reaction information.
3. Construct useful numerical and categorical features.
4. Create drug interaction and graph-based risk features.
5. Compare Logistic Regression, Decision Tree, Random Forest, and Gradient Boosting models.
6. prevent data leakage during graph feature construction and model selection.
7. Select the best model according to mean cross-validation ROC-AUC.
8. Tune the selected model using `RandomizedSearchCV`.
9. Evaluate the final model once on a held-out test set.
10. Save the complete trained pipeline as a reusable `.pkl` artifact.
11. Create an independent inference workflow for new database records.
12. Store predicted labels and probabilities in the SQLite database.

3 Dataset and Relational Database

The processed model dataset is stored at:

```
data/processed/model_dataset.csv
```

The primary SQLite database used by the automated prediction pipeline is:

```
database/faers.db
```

The implementation can also search several alternative database paths when training the model. This makes the project more robust to differences in local directory structure.

The relational database contains report information across tables such as:

- `reports`: report-level information;
- `patients`: patient age, weight, and sex;
- `drugs`: drugs associated with each report;
- `reactions`: reported adverse reactions;
- `predictions`: predictions generated by the inference pipeline.

The common identifier used to connect these entities is `report_id`.

4 Task Definition and Input Features

4.1 Target Variable

The target column is `serious`. It is converted to an integer and used as the response variable for the binary classification models.

The training data are imbalanced. The observed class distribution in the training set was:

Table 1: Class distribution in the final training set

Class	Meaning	Proportion
0	Non-serious	0.2243
1	Serious	0.7757

Because approximately 77.57% of the training samples belong to the serious class, stratification and imbalance-aware evaluation metrics were important. The models that support class weighting were configured with `class_weight="balanced"`.

4.2 Base Features

Seven base input features were used:

1. `patient_age`
2. `patient_weight`
3. `num_drugs`

4. `num_suspect_drugs`
5. `num_reactions`
6. `patient_sex_male`
7. `patient_sex_unknown`

Patient sex is normalized into the categories `female`, `male`, and `unknown`. One-hot encoding is then applied with female as the reference category, resulting in the two indicator columns `patient_sex_male` and `patient_sex_unknown`.

4.3 Graph-Based Features

Three additional features were constructed from the drug co-occurrence graph:

1. `max_drug_risk`
2. `avg_drug_risk`
3. `sum_drug_risk`

Consequently, the final model contains ten predictive features in total.

5 Graph-Based Drug Risk Feature Engineering

5.1 Drug Co-occurrence Graph

A weighted, undirected drug co-occurrence graph was constructed using NetworkX. Each graph node represents a normalized drug name. Two drugs are connected when they occur together in at least one adverse event report.

For a report containing the unique drug set

$$D_r = \{d_1, d_2, \ldots, d_k\},$$

all unordered pairs of drugs are generated. The edge weight between drugs d_i and d_j is the number of reports in which the pair occurs together:

$$w_{ij} = \sum_r \mathbb{I}(d_i \in D_r \wedge d_j \in D_r).$$

Drug names are converted to uppercase and surrounding whitespace is removed before graph construction. This reduces inconsistencies caused by capitalization and formatting.

5.2 PageRank Risk Score

Weighted PageRank is applied to the graph. A drug with a larger PageRank score is more central in the observed drug co-occurrence structure. The PageRank values are normalized to a range from 0 to 100:

$$\text{Risk}(d) = \frac{\text{PageRank}(d)}{\max_{v \in V} \text{PageRank}(v)} \times 100.$$

The normalized score is a graph centrality-based risk indicator. It does not directly represent a clinical probability of harm; rather, it summarizes the structural importance of a drug within the co-occurrence network used by this project.

For every report, the drug-level scores are aggregated as follows:

$$\begin{aligned}\text{MaxRisk}(r) &= \max_{d \in D_r} \text{Risk}(d),\\ \text{AvgRisk}(r) &= \frac{1}{|D_r|} \sum_{d \in D_r} \text{Risk}(d),\\ \text{SumRisk}(r) &= \sum_{d \in D_r} \text{Risk}(d).\end{aligned}$$

If a report has no recognized drug or contains a drug not observed during graph fitting, the unavailable graph risk contribution is set to zero.

5.3 Leakage Prevention

Graph features depend on relationships among reports. Therefore, constructing the graph on the complete dataset before cross-validation could expose validation or test-set structure to the training process.

To prevent this problem, the project follows a fold-safe strategy:

1. In each cross-validation fold, the graph is built only from the training portion of that fold.
2. Drug risk scores are calculated only from the fold's training reports.
3. The validation fold is transformed using the training-derived risk map.
4. During final training, the graph is fitted only on the outer training set.
5. The held-out test set is transformed using risk scores learned exclusively from the outer training set.
6. Drugs unseen during training receive a graph risk score of zero.

During hyperparameter tuning, graph construction is implemented inside a custom Scikit-learn transformer named `GraphRiskFeatureTransformer`. Since the transformer is part of the Pipeline, Scikit-learn refits it independently inside every cross-validation fold. This makes model selection and tuning leakage-free.

6 Training, Validation, and Test Strategy

An outer stratified split was first applied:

- **80%**: outer training data;
- **20%**: held-out test data.

The split was generated using:

```
train_df, test_df = train_test_split(
    df,
    test_size=0.2,
    random_state=42,
    stratify=df["serious"].astype(int),
)
```

The held-out test set was not used for model comparison, hyperparameter selection, or graph fitting. Model comparison and hyperparameter tuning were performed only on the outer training portion.

Within the training set, 5-fold stratified cross-validation was used:

```
StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42,
)
```

Stratification preserves approximately the same class distribution in each fold.

7 Candidate Models

The following four classification algorithms were evaluated:

1. **Logistic Regression**: a linear and interpretable baseline model;
2. **Decision Tree**: a non-linear tree-based classifier;
3. **Random Forest**: an ensemble of decision trees with reduced variance;
4. **Gradient Boosting**: a sequential ensemble that corrects previous errors.

These models provide a useful comparison between linear, single-tree, bagging, and boosting approaches.

The primary model-selection metric was mean cross-validation ROC-AUC. ROC-AUC measures the model's ability to rank serious cases above non-serious cases across all possible classification thresholds. This property makes it more informative than accuracy alone for an imbalanced binary classification task.

8 Cross-Validation Model Comparison

Table 2 presents the mean performance of each candidate model across five fold-safe cross-validation folds. The values in parentheses show the population standard deviation across folds.

Table 2: Fold-safe 5-fold cross-validation model comparison

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
Random Forest	0.7488 ± 0.0119	0.9067 ± 0.0054	0.7536 ± 0.0118	0.8231 ± 0.0091	$\mathbf{0.8237 \pm 0.0119}$
Gradient Boosting	$\mathbf{0.8049 \pm 0.0043}$	0.8158 ± 0.0024	$\mathbf{0.9669 \pm 0.0038}$	$\mathbf{0.8849 \pm 0.0026}$	0.8107 ± 0.0128
Logistic Regression	0.5494 ± 0.0084	0.8794 ± 0.0073	0.4857 ± 0.0092	0.6257 ± 0.0086	0.6726 ± 0.0127
Decision Tree	0.7577 ± 0.0116	0.8442 ± 0.0034	0.8432 ± 0.0166	0.8436 ± 0.0089	0.6501 ± 0.0087

Gradient Boosting achieved the highest mean accuracy, recall, and F1-score during model comparison. However, the predefined selection metric was mean cross-validation ROC-AUC. According to this metric, Random Forest achieved the best score:

$$\text{Mean CV ROC-AUC}_{\text{RF}} = 0.8237.$$

Therefore, the model selected for hyperparameter tuning and final training was:

Selected model: Random Forest

The mean number of graph drugs available in each cross-validation training graph was approximately 9068.8. The model comparison test flag was set to false, confirming that the held-out test set was not involved in selecting the final algorithm.

The aggregate and fold-level results were saved respectively to:

```
data/processed/model_comparison.csv
data/processed/model_comparison_fold_results.csv
```

9 Hyperparameter Tuning

After Random Forest was selected, its hyperparameters were optimized using `RandomizedSearchCV`. The tuning procedure used:

- 5-fold `StratifiedKFold`;
- ROC-AUC as the optimization score;
- 30 randomized configurations for Random Forest;
- `random_state=42`;
- parallel execution through `n_jobs=-1`.

The final Scikit-learn Pipeline contains:

1. `GraphRiskFeatureTransformer`;
2. `SimpleImputer(strategy="median")`;
3. `StandardScaler`;
4. `RandomForestClassifier`.

Its general structure is:

```
Pipeline(
    steps=[
        (
            "graph_features",
            GraphRiskFeatureTransformer(
                drugs_df=drugs_df,
                base_features=BASE_FEATURES,
                graph_features=GRAPH_FEATURES,
            ),
        ),
        ("imputer", SimpleImputer(strategy="median")),
```

```
        ("scaler", StandardScaler()),
        ("classifier", RandomForestClassifier(...)),
    ]
)
```

The best hyperparameters found by the randomized search were:

Table 3: Best Random Forest hyperparameters

Hyperparameter	Selected value
`n_estimators`	500
`max_depth`	20
`min_samples_split`	2
`min_samples_leaf`	1
`max_features`	`sqrt`

The best mean validation ROC-AUC obtained during hyperparameter tuning was:

$$\text{Best tuning CV ROC-AUC} = 0.8336.$$

This score is higher than the initial Random Forest comparison ROC-AUC of 0.8237, indicating that tuning improved cross-validation ranking performance.

The hyperparameter search required approximately 167.50 seconds. Search results and selected parameters were saved in the processed-data directory.

10 Final Model Evaluation

After hyperparameter selection, the optimized pipeline was fitted on the complete outer training set and evaluated once on the held-out test set.

10.1 Evaluation Metrics

The following metrics were calculated:

$$\begin{aligned}
\text{Accuracy} &= \frac{TP + TN}{TP + TN + FP + FN}, \\
\text{Precision} &= \frac{TP}{TP + FP}, \\
\text{Recall} &= \frac{TP}{TP + FN}, \\
F_1 &= 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}.
\end{aligned}$$

ROC-AUC was calculated using predicted probabilities for the serious class.

10.2 Held-Out Test Results

Table 4: Final Random Forest performance on the held-out test set

Metric	Value
Accuracy	0.8048
Precision	0.8918
Recall	0.8517
F1-score	0.8713
ROC-AUC	0.8484

The final test ROC-AUC of 0.8484 indicates that the model has a useful ability to rank serious reports above non-serious reports. Precision of 0.8918 means that approximately 89.18% of reports predicted as serious were actually serious. Recall of 0.8517 means that the model detected approximately 85.17% of all serious reports in the test set.

The final model required approximately 2.78 seconds for fitting on the complete training set and 0.16 seconds for test prediction.

10.3 Confusion Matrix

The final confusion matrix was:

$$\begin{bmatrix} TN & FP \\ FN & TP \end{bmatrix} = \begin{bmatrix} 376 & 209 \\ 300 & 1723 \end{bmatrix}.$$

Therefore:

- **376** non-serious reports were correctly classified;
- **209** non-serious reports were incorrectly classified as serious;
- **300** serious reports were incorrectly classified as non-serious;
- **1723** serious reports were correctly classified.

The test set contained 2608 reports in total.

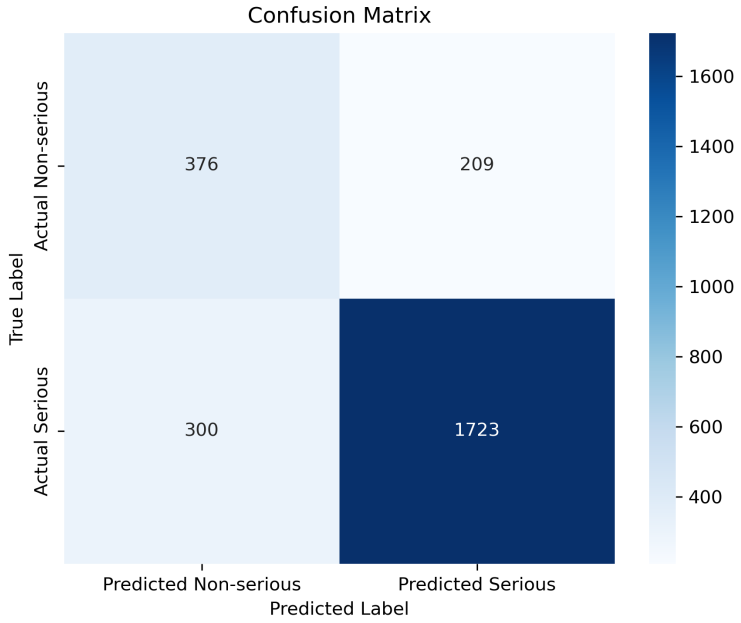

Figure 1: Confusion matrix of the final Random Forest model

10.4 ROC Curve

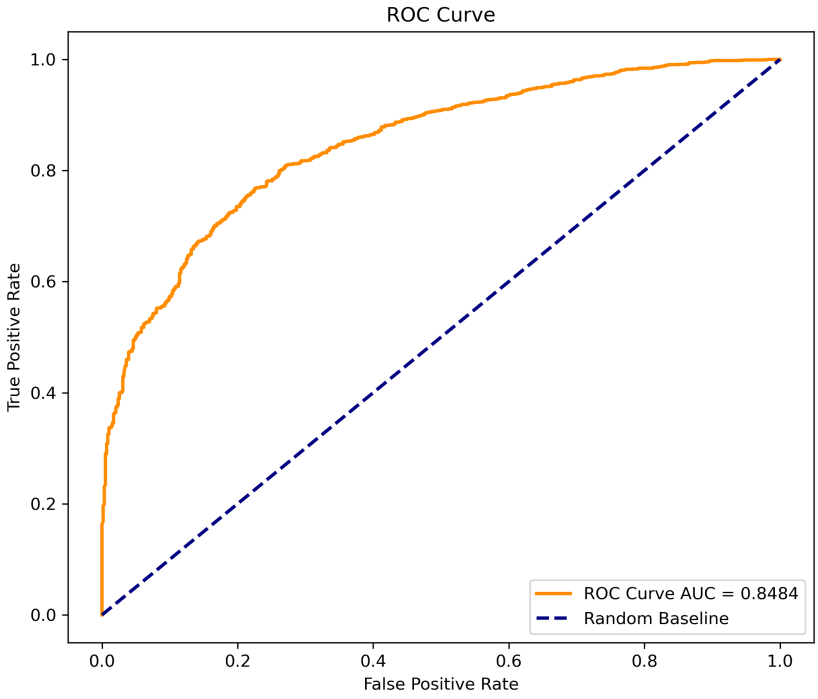

Figure 2: ROC curve of the final model with test ROC-AUC of 0.8484

11 Feature Importance

Since the selected model is a Random Forest, feature importance values were extracted from the classifier's `feature_importances_` attribute.

Table 5: Random Forest feature importance values

Rank	Feature	Importance
1	`sum_drug_risk`	0.1823
2	`avg_drug_risk`	0.1570
3	`patient_age`	0.1505
4	`max_drug_risk`	0.1232
5	`num_drugs`	0.0968
6	`num_reactions`	0.0901
7	`num_suspect_drugs`	0.0885
8	`patient_weight`	0.0722
9	`patient_sex_male`	0.0269
10	`patient_sex_unknown`	0.0125

The three graph-based features together account for approximately:

$$0.1823 + 0.1570 + 0.1232 = 0.4625,$$

or 46.25% of the total Random Forest feature importance. This suggests that the graph-derived representation provides substantial predictive information to the model. The most influential individual variable was `sum_drug_risk`, followed by `avg_drug_risk` and `patient_age`.

Feature importance describes how strongly the fitted Random Forest used each variable for splitting. It does not prove a causal relationship between a feature and adverse event seriousness.

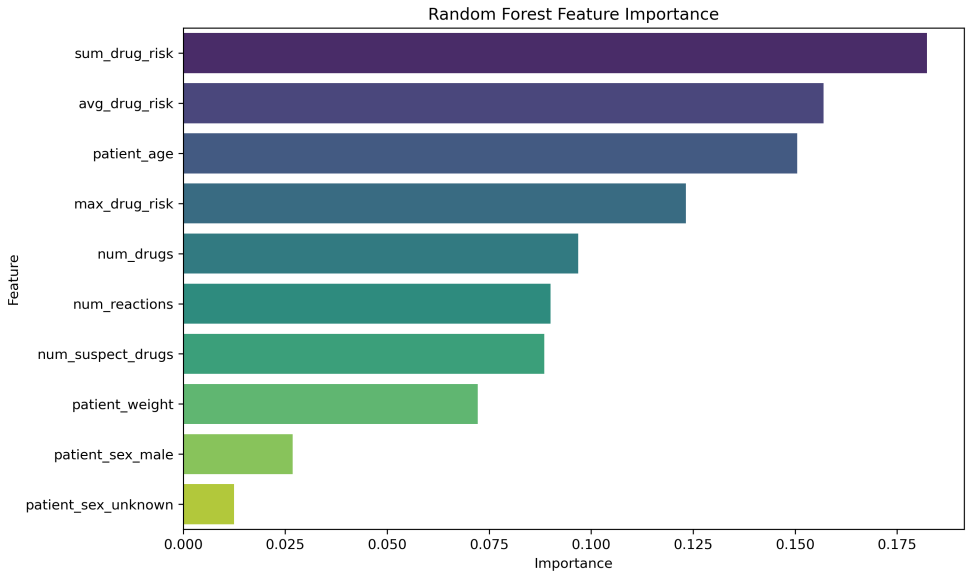

Figure 3: Feature importance values of the final Random Forest classifier

12 Training Pipeline Automation with Prefect

12.1 Reason for Selecting Prefect

Prefect was selected as the workflow orchestration tool. It provides structured task definitions, dependency management, retries, execution states, and logging.

Compared with a simple sequential `run_pipeline.py` script, Prefect provides better observability and explicit workflow dependencies. Compared with GitHub Actions, Prefect is more directly focused on data workflows and can be used locally without requiring every run to be triggered through a remote repository event.

The main benefits of Prefect in this project are:

- explicit dependencies between workflow stages;
- automatic retries for temporary failures;
- detailed task and flow logs;
- parallel execution of independent tasks;
- separation of training and prediction workflows;
- easier future scheduling and deployment.

Possible shortcomings include the need to install and configure an additional orchestration framework, increased complexity compared with a basic Python script, and the need for extra infrastructure if a centralized Prefect server or cloud deployment is required.

12.2 Training Flow Tasks

The training flow is named:

```
FAERS Drug Risk Analysis Pipeline
```

It contains the following Prefect tasks:

1. **Load Relational Database**: creates and loads the SQLite relational database.
2. **Preprocess Dataset**: prepares the model dataset.
3. **Generate Advanced Interaction Features**: creates interaction-related features.
4. **Graph Risk Scoring**: calculates graph-based drug risk values.
5. **Build Interaction Network**: creates the interaction network visualization.
6. **Compare Models**: performs fold-safe cross-validation and saves model comparison results.
7. **Train Final Model**: selects, tunes, evaluates, and saves the final model.

Every task is configured with two retries and a five-second retry delay:

```
@task(name="Task Name", retries=2, retry_delay_seconds=5)
```

12.3 Training Flow Dependencies

The dependency structure is summarized below:

1. Database creation runs first.
2. Preprocessing and advanced interaction feature generation can begin after database creation.
3. Graph risk scoring waits for interaction feature generation.
4. Network visualization waits for graph risk scoring.
5. Model comparison waits for preprocessing.
6. Final model training waits for model comparison.

This structure allows independent branches of the workflow to execute concurrently while preserving the required ordering within each branch.

A simplified view of the flow is:

```
Load Relational Database
    |---> Preprocess Dataset
    | |---> Compare Models
    | |---> Train Final Model
    |
    |---> Generate Advanced Interaction Features
            |---> Graph Risk Scoring
                     |---> Build Interaction Network
```

Model comparison and training independently apply their fold-safe graph feature logic to the preprocessed model data. Therefore, the model-selection branch does not depend on a graph file precomputed from the complete dataset.

13 Final Model Artifact and Saved Outputs

The complete fitted model Pipeline was serialized using Joblib and saved as:

```
models/final_model.pkl
```

This artifact contains:

- the fitted graph-risk transformer and training-derived risk map;
- the median imputer;
- the standardization step;
- the optimized Random Forest classifier.

Saving the entire Pipeline ensures that the same transformations used during training are automatically applied during inference.

Important generated outputs include:

Output path	Description
`models/final_model.pkl`	Complete fitted prediction Pipeline

Output path	Description
`data/processed/model_metrics.json`	Test metrics, parameters, versions, and runtime information
`data/processed/model_comparison.csv`	Aggregate model comparison results
`data/processed/model_comparison_fold_results.csv`	Fold-level validation metrics
`data/processed/feature_importances.csv`	Final feature importance values
`data/processed/train_only_graph_drug_risk.csv`	Training-derived graph risk scores
`reports/figures/confusion_matrix.png`	Confusion matrix visualization
`reports/figures/roc_curve.png`	ROC curve visualization
`reports/figures/feature_importances.png`	Feature importance visualization

14 Prediction Pipeline

A separate Prefect flow named `FAERS Drug Risk Prediction Pipeline` was implemented for inference. It does not retrain or tune the model. Instead, it loads the saved artifact and generates predictions for records in the relational database.

14.1 Prediction Tasks

The prediction workflow includes five major tasks:

1. **Load Prediction Data**: reads report and patient information from SQLite and computes missing report-level counts when required.
2. **Preprocess Prediction Data**: converts numeric fields, handles missing values, normalizes patient sex, and creates the expected dummy variables.
3. **Load Trained Model**: searches candidate artifact paths and loads the saved model with Joblib.
4. **Generate Predictions**: produces predicted labels and serious-class probabilities.
5. **Save Predictions to Database**: creates the predictions table if needed and saves the generated results.

The model-loading task prioritizes:

```
models/final_model.pkl
```

which is the artifact produced by the final training process.

14.2 Prediction Input Preparation

The inference workflow prepares the exact inputs expected by the serialized Pipeline:

```
report_id
patient_age
patient_weight
num_drugs
num_suspect_drugs
num_reactions
patient_sex_male
```

```
patient_sex_unknown
```

The graph-based features do not need to be manually included in this DataFrame because the serialized `GraphRiskFeatureTransformer` creates them internally using its stored training-derived risk map and the drug records connected to each `report_id`.

If report-level count columns are not already available, the prediction pipeline calculates them from the related tables. For example, `num_drugs` and `num_reactions` are obtained by grouped row counts. The implementation also attempts to detect a drug-role column to calculate `num_suspect_drugs`.

14.3 Predicted Values

For each report, the pipeline generates:

- `predicted_label`: binary predicted class;
- `predicted_probability`: estimated probability of the serious class;
- `model_path`: path of the artifact used for inference;
- `predicted_at`: ISO-formatted prediction timestamp.

When the model provides `predict_proba`, the probability of class 1 is used. A fallback based on `decision_function` is also implemented for models that do not provide class probabilities.

15 Saving Predictions Back to SQLite

Predictions are stored in the `predictions` table. If this table does not exist, the workflow creates it automatically using the following conceptual schema:

Table 7: Schema of the SQLite predictions table

Column	SQLite type
`report_id`	TEXT
`predicted_label`	INTEGER
`predicted_probability`	REAL
`model_path`	TEXT
`predicted_at`	TEXT

Before new values are inserted, existing prediction rows for the same report identifiers are deleted. The new prediction rows are then appended. This prevents duplicate active predictions for report identifiers processed in the current run while allowing the workflow to update previous results.

This fulfills the project requirement that generated predictions must be saved back to the database for later access and analysis.

16 End-to-End Execution

The training pipeline can be executed from its Python file with a single command:

```
python path/to/training_pipeline.py
```

Similarly, the independent prediction flow can be executed using:

```
python path/to/prediction_pipeline.py
```

When the files are executed directly, their corresponding Prefect flows are called through the standard Python entry point:

```
if __name__ == "__main__":
    faers_pipeline()
```

and:

```
if __name__ == "__main__":
    prediction_pipeline()
```

Thus, both training and prediction workflows can be started with a single command or function call.

17 Reproducibility

The following measures support reproducibility:

- `random_state=42` is used for splitting, cross-validation, randomized search, and supported models.
- Stratified splitting preserves class distributions.
- The held-out test set is excluded from model selection.
- Graph construction is repeated independently inside each training fold.
- Hyperparameters and complete search results are saved.
- Aggregate and fold-level model comparison results are stored as CSV files.
- Final metrics and runtime information are stored in JSON format.
- The entire fitted preprocessing and classification Pipeline is serialized.
- Prefect logs task execution and retries failed stages.

The main software versions recorded during final model training were:

Table 8: Main software versions

Library	Version
Scikit-learn	1.9.0
Pandas	2.3.0
NetworkX	3.6.1

For reliable artifact loading, the prediction environment should use compatible package versions, particularly for Scikit-learn and custom Python classes included in the serialized Pipeline.

18 Discussion

The experimental results show that the four candidate algorithms have different strengths. Gradient Boosting achieved very high recall during initial cross-validation, while Random Forest obtained the strongest mean ROC-AUC. Since ROC-AUC was established as the main model-selection criterion, Random Forest was correctly chosen for the final tuning stage.

After tuning, the selected model achieved a cross-validation ROC-AUC of 0.8336 and a held-out test ROC-AUC of 0.8484. Its final precision of 0.8918 and recall of 0.8517 produced an F1-score of 0.8713. These results indicate a good balance between correctly identifying serious reports and limiting false serious predictions.

The graph-based variables were particularly influential. Their combined Random Forest importance was approximately 46.25%. This result supports the idea that drug co-occurrence relationships contain information not fully represented by basic patient and count variables.

An important methodological strength is that the graph was not generated once from all available data and then reused during validation. Instead, graph computation was limited to the appropriate training subset at every stage. This reduces the risk of optimistic evaluation caused by hidden validation or test information.

The use of Prefect makes the project modular and repeatable. Training and prediction responsibilities are separated, and the prediction flow uses a fixed saved model rather than retraining on inference records.

19 Limitations

The current system has several limitations:

- FAERS is a spontaneous reporting system and may contain incomplete, duplicated, inconsistent, or biased reports.
- A model prediction does not establish that a drug caused an adverse event.
- PageRank centrality is a structural graph measure and should not be interpreted as a validated clinical drug risk probability.
- Drug-name normalization currently relies mainly on whitespace removal and uppercase conversion; synonyms and brand/generic variations may remain separate.
- Unseen drugs receive a graph score of zero, which may not accurately reflect their real risk.
- The class distribution is imbalanced, with serious reports representing most training records.
- The default decision threshold is used; a clinically motivated threshold optimization procedure was not included.
- The prediction workflow currently loads available report records and replaces predictions for matching identifiers rather than exclusively filtering records that have never been predicted.
- Random Forest feature importance can be biased and does not provide causal interpretation.

- The model may require retraining if the underlying FAERS data distribution changes over time.

20 Future Work

Possible future improvements include:

- integrate MLflow for experiment tracking, artifact management, and model versioning;
- evaluate XGBoost, LightGBM, CatBoost, and calibrated classifiers;
- optimize the probability threshold according to clinical or operational costs;
- report Precision–Recall AUC in addition to ROC-AUC;
- apply more advanced drug-name normalization using RxNorm or another medical terminology resource;
- create additional graph features such as weighted degree, betweenness centrality, community membership, and embeddings;
- use SHAP values for local and global model interpretation;
- add a model-version field and unique constraints to the predictions table;
- filter the prediction query to process only new or modified reports;
- schedule periodic inference runs through Prefect deployments;
- expose predictions through a REST API using FastAPI or Spring Boot;
- add automated data-quality and model-drift monitoring.

MLflow was considered as the bonus model-management option but was not required for the current functional end-to-end implementation. It can be integrated later to track parameters, metrics, software environments, artifacts, and model versions.

21 Conclusion

In Phase 3, a complete end-to-end machine learning solution was developed for predicting the seriousness of FDA FAERS adverse event reports. The solution combines seven conventional patient and report features with three graph-based drug risk features generated from a weighted drug co-occurrence network.

Four candidate models were evaluated through leakage-free 5-fold stratified cross-validation. Random Forest obtained the highest mean cross-validation ROC-AUC of 0.8237 and was selected for hyperparameter tuning. The optimized model used 500 estimators, a maximum depth of 20, `sqrt` feature sampling, a minimum leaf size of 1, and a minimum split size of 2.

On the held-out test set, the final model achieved an accuracy of 0.8048, precision of 0.8918, recall of 0.8517, F1-score of 0.8713, and ROC-AUC of 0.8484. Graph-derived variables represented approximately 46.25% of total Random Forest feature importance, indicating that drug co-occurrence structure contributed substantially to the predictions.

Data leakage was controlled by fitting graph risk scores only on the relevant training data inside each cross-validation fold and on the final outer training split. The test set was not used for model selection or tuning.

Finally, separate Prefect workflows were implemented for training and prediction. The training workflow saves the complete fitted Pipeline as `models/final_model.pkl`. The inference workflow loads this artifact, processes database records, generates predicted labels and probabilities, and stores the results in the SQLite `predictions` table. Therefore, the project satisfies the requirements of model development, evaluation, automation, reusable inference, and database prediction storage.